Revisiting the Performance of Deep Learning-Based Vulnerability Detection on Realistic Datasets

Partha Chakraborty, *Student Member, IEEE*, Krishna Kanth Arumugam, Mahmoud Alfadel, *Member, IEEE*, Meiyappan Nagappan, and Shane McIntosh, *Senior Member, IEEE*

***Abstract*—The impact of software vulnerabilities on everyday software systems is concerning. Although deep learning-based models have been proposed for vulnerability detection, their reliability remains a significant concern. While prior evaluation of such models reports impressive recall/F1 scores of up to 99%, we find that these models underperform in practical scenarios, particularly when evaluated on the entire codebases rather than only the fixing commit. In this paper, we introduce a comprehensive dataset (*Real-Vul*) designed to accurately represent real-world scenarios for evaluating vulnerability detection models. We evaluate DeepWukong, LineVul, ReVeal, and IVDetect vulnerability detection approaches and observe a surprisingly significant drop in performance, with precision declining by up to 95 percentage points and F1 scores dropping by up to 91 percentage points. A closer inspection reveals a substantial overlap in the embeddings generated by the models for vulnerable and uncertain samples (non-vulnerable or vulnerability not reported yet), which likely explains why we observe such a large increase in the quantity and rate of false positives. Additionally, we observe fluctuations in model performance based on vulnerability characteristics (e.g., vulnerability types and severity). For example, the studied models achieve 26 percentage points better F1 scores when vulnerabilities are related to information leaks or code injection rather than when vulnerabilities are related to path resolution or predictable return values. Our results highlight the substantial performance gap that still needs to be bridged before deep learning-based vulnerability detection is ready for deployment in practical settings. We dive deeper into why models underperform in realistic settings and our investigation revealed overfitting as a key issue. We address this by introducing an augmentation technique, potentially improving performance by up to 30%. We contribute (a) an approach to creating a dataset that future research can use to improve the practicality of model evaluation; (b) *Real-Vul*– a comprehensive dataset that adheres to this approach; and (c) empirical evidence that the deep learning-based models struggle to perform in a real-world setting.**

***Index Terms*—Vulnerability, security, machine learning, deep learning.**

Manuscript received 24 August 2023; revised 13 June 2024; accepted 26 June 2024. Date of publication 5 July 2024; date of current version 15 August 2024. Recommended for acceptance by L. Ma. *(Corresponding author: Partha Chakraborty.)*

The authors are with David R. Cheriton School of Computer Science, University of Waterloo, Waterloo, ON N2L 3G1, Canada (e-mail: p9chakra@uwaterloo.ca; kkarumug@uwaterloo.ca; malfadel@uwaterloo.ca; mei.nagappan@uwaterloo.ca; shane.mcintosh@uwaterloo.ca).

Digital Object Identifier 10.1109/TSE.2024.3423712

I. Introduction

SOFTWARE vulnerabilities have a large negative impact on the software systems. Identifying and addressing vulnerabilities in complex systems with multiple interconnected components has only exacerbated the problem, making a comprehensive and systematic approach necessary. Machine learning models that are trained using Deep Neural Networks (DNNs) have shown promise in identifying software vulnerabilities [3], [4].

However, the reliability of these models in detecting vulnerabilities in real-world scenarios depends on the evaluation methodology and dataset. Biases that can affect model performance can arise from various sources, such as the manner in which the dataset is generated and labeled. The generalizability of a model may suffer if the dataset on which it is trained is biased. For instance, synthetic datasets, such as SARD [38], are artificially created using fuzz techniques and evolutionary algorithms and are extensively used to evaluate the effectiveness of deep learning models in detecting software vulnerabilities [7], [33], [52]. It is unclear the extent to which the vulnerabilities in these synthetic datasets capture the full range of complexities and variations that are found in real-world vulnerabilities.

Moreover, these studies classified explicitly identified vulnerable functions as "vulnerable" and all remaining functions as "non-vulnerable". However, this binary classification overlooks the possibility that some functions might be vulnerable yet undetected. Therefore, for the purposes of our study, we will describe these functions with a more nuanced term, "uncertain," to acknowledge the inherent ambiguity in their vulnerability status.

On the other hand, there do exist real-world datasets that contain vulnerabilities from real software systems (e.g., BigVul [16], ReVeal [4]); however, these datasets tend to **(i)** only include a sample of code from a substantially larger codebase, i.e., the vulnerable and uncertain (non-vulnerable or vulnerability not reported yet) samples are extracted from vulnerability-related commits, and hence, the dataset does not reflect a realistic setting where vulnerability detection models are deployed to scan an entire project; and **(ii)** suffer from label inconsistency wherein the same sample is marked as both vulnerable and uncertain (see Section III for detailed explanation). Consequently, we conjecture that models that are trained and evaluated using such datasets may not perform well when applied in real-world settings.

0098-5589 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:03:49 UTC from IEEE Xplore. Restrictions apply.

Therefore, in this paper, we introduce *Real-Vul*, a new vulnerability detection dataset designed to accurately represent the realistic settings in which these models would be deployed. By "**realistic settings**," we refer to evaluation environments that closely mirror real-world conditions, including the distribution of vulnerable and uncertain samples. This approach ensures that the models are tested under conditions that reflect their actual usage scenarios.

Real-Vul differs from prior existing datasets in that it includes entire codebases in which vulnerable code exists, representing a more realistic setting in which vulnerability detection models would be applied. *Real-Vul* also accounts for the label inconsistency problem by comparing the hash of vulnerable and uncertain code segments.

To re-evaluate the performance of deep learning-based models in our more realistic setting, we apply four *state-of-the-art* models (i.e., DeepWukong [7], LineVul [20], ReVeal [4], and IVDetect [30]) to *Real-Vul* dataset. First, we train and evaluate these models on their original datasets, i.e., the SARD [38], BigVul [16], ReVeal [16], and IVDetect [30] datasets, respectively. The DeepWukong and LineVul models achieve notably high precision (87% - 96%) and F1 scores (90% - 93%), while ReVeal and IVDetect achieves precision and F1 scores of 29% - 39% and 78% - 83%, respectively. However, when we evaluate these models on the *Real-Vul* dataset, we observe a substantial decrease in performance, e.g., precision and F1 score decreases of up to 95 and 91 percentage points, respectively.

To understand why these models produce a large number of false positives, we visualize their embeddings for vulnerable and uncertain samples in a two-dimensional space using t-SNE [46]. We find that these models struggle to establish a clear distinction between the two classes, resulting in inaccurate identification of vulnerabilities. For instance, models like LineVul heavily rely on lexical relationships between tokens and as such, encounter difficulties in accurately capturing the complex nature of vulnerabilities in real-world software.

To pinpoint why models underperform in realistic settings, we conducted an in-depth manual analysis of false positives. We found that overfitting, where models predict based on specific tokens, was a significant issue. To counteract this, we implemented an augmentation technique that has the potential to enhance performance by up to 30%. Furthermore, to understand the capability of the studied models in identifying different types of vulnerabilities, we stratify our model evaluation by vulnerability type. Our findings indicate that the models achieve 26 percentage points higher F1 scores in detecting vulnerability from specific types, such as information leaks and code injection, and struggle to detect vulnerabilities from other types, like path resolution and predictable return values. Additionally, we find that the models perform poorly in identifying high-severity vulnerabilities, showing further challenges for these models in identifying important vulnerabilities.

Contributions. This paper makes the following contributions:

- We introduce *Real-Vul*, a new vulnerability detection dataset that aims to address the practical limitations of prior datasets.
- We re-evaluate the performance of existing *state-of-the-art* vulnerability detection approaches on the *Real-Vul* dataset. Our evaluation provides a more accurate representation of the performance of these models when used in realistic settings.
- We characterize the limitations of the studied approaches in terms of:
  - ⋆ their embeddings, which show distinct overlaps across vulnerable and uncertain samples, suggesting models' inability to clearly distinguish vulnerable codes from uncertain codes.
  - ⋆ tendencies to be unable to detect vulnerabilities belonging to particular types and severities. Stratifying model performance by vulnerability type and severity allows targeted improvements and efficient resource allocation in addressing distinct security risks.

Fostering open science: To foster future advances to modeling approaches for vulnerability detection, we release our *Real-Vul* dataset, and we also make the scripts used for our experiments publicly available.[1]

II. Limitations of Existing Datasets

As software systems become more complex and larger, the potential for security vulnerabilities also increases. Therefore, it is crucial to have tools to discover these vulnerabilities. Machine learning models have proven to be effective in understanding code and detecting vulnerabilities. However, obtaining and evaluating large datasets in a realistic setting [4], [16], [54] remains challenging. Existing studies on vulnerability detection [16], [54] rely on datasets that are based on various criteria and may not reflect reality accurately. These datasets can be classified into three categories: synthetic, oracle-based, and real-world datasets. In this section, we describe each category and their limitations.

Synthetic dataset (e.g., SARD [38]). The Software Assurance Reference Dataset (SARD) is an example of a synthetic dataset. It contains a vast number of artificially produced C/C++ programs containing numerous security vulnerabilities. SARD has been created through automated methods like fuzzing or genetic algorithms. The SARD dataset's limitation lies in containing only artificially generated vulnerabilities, which may not accurately represent real-world software vulnerabilities. This dataset does not reflect the complexities and variations found in actual code written by developers, potentially limiting researchers from evaluating their models in a realistic setting.

Oracle-based dataset (e.g., D2A [53]). D2A is an example of an Oracle-based dataset. Unlike synthetic datasets, oracle-based datasets rely on third-party sources such as static analysis tools to provide labels for collected data samples. Although such datasets offer more complexity than synthetic datasets, they may not fully represent real-world vulnerabilities due to oversimplification. The inaccuracy of the labeling heavily affects the dataset's reliability which threatens the usability of the

[1]https://zenodo.org/record/8206635

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:03:49 UTC from IEEE Xplore. Restrictions apply.

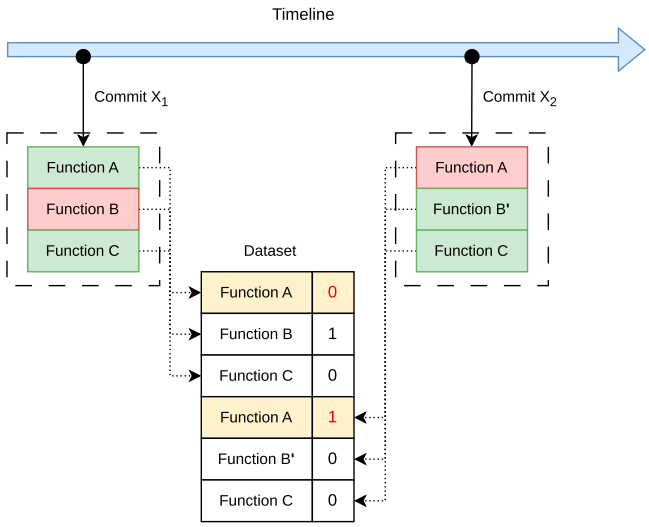


Fig. 1. Example of label inconsistency.

vulnerability detection models trained using this type of dataset. **Real-world dataset (e.g., Big-Vul [16], Devign [54], ReVeal [4]).** Big-Vul and ReVeal datasets are examples of real-world datasets for vulnerability detection. They are more diverse than synthetic and oracle-based datasets; real-world datasets are generated using data available in issue-tracking systems and source code repositories.

Big-Vul dataset [16] is composed of C/C++ functions collected from 348 open-source GitHub projects spanning from 2002 to 2019. The Devign dataset has been curated from Linux Kernel, QEMU, Wireshark, and FFmpeg projects, whereas the ReVeal dataset [4] is collected from the Chromium and Debian project. The above-mentioned datasets still have limitations. First, such datasets fail to entirely capture vulnerability detection in a realistic setting where a comprehensive scan of the entire source code of a project would be performed. In realistic setting, the vulnerable functions are rare compared to uncertain ones, leading to an imbalance in their occurrence during scans. Conversely, prior datasets typically categorize altered or post-fix functions as uncertain and unchanged or pre-fix functions as vulnerable, resulting in an unrealistic nearly equal ratio of uncertain to vulnerable functions. Moreover, since vulnerable functions are rare in the real world, these datasets tend to be smaller and less diverse. Training on such a limited dataset will result in a biased model. Additionally, since the test dataset will also be biased, it will be challenging to detect these biases in the model.

Second, these datasets may suffer from *label inconsistency*. Fig. 1 elaborates further on the problem of label inconsistency. In the figure, consider two vulnerability-fixing commits, X_1 and X_2, with X_1 occurring before X_2. In X_1, the vulnerable function is Function B (note that after fixing it, it became Function B'). Following the data collection policy of previous studies (e.g., ReVeal [4]), Function B would be labeled as vulnerable, while the other unchanged functions (Function A and C) would be labeled as uncertain. However, as we progress further in the timeline, we encounter commit X_2, which fixes the vulnerability in "Function A". Now, as per the aforementioned policy, "Function A" would be labeled as vulnerable, while Function B'and C would be labeled as uncertain. Consequently, the dataset will contain two entries for Function A, where in one case, it is marked as vulnerable and in the other as uncertain. This discrepancy in labeling creates the label inconsistency issue. In fact, we verified the presence of label inconsistency in existing datasets and found that 15% vulnerable samples in the ReVeal dataset have been listed as uncertain samples.

Due to the aforementioned limitations, current deep learning models for vulnerability detection (e.g., [4], [7], [20], [32], [54]) could result in unrealistic evaluations compared to real-world usage. Additionally, the reported model performance might be either exaggerated or understated. Therefore, in this study, we build *Real-Vul*, a new dataset that takes into consideration (1) a realistic setting where a comprehensive scan of the entire source code of a project would be performed; and (2) label consistency. The next section presents *Real-Vul* and the process we follow to create it.

III. *Real-Vul* Dataset

In this section, we introduce our proposed dataset, named *Real-Vul*, which addresses the limitations of existing vulnerability detection datasets (Section II). Table I provides an overview of the projects that are incorporated in *Real-Vul*.

Project selection. *Real-Vul* includes separate training and testing datasets designed for training and evaluating vulnerability detection models. To ensure high-quality vulnerable samples, we carefully select projects from the Big-Vul dataset [16] based on the number of vulnerabilities in each project. Big-Vul dataset comprises real-world vulnerable C/C++ functions. It also provides rich metadata for vulnerabilities, including line numbers, vulnerability-fixing commit hashes, CVE IDs, severity rankings, and summaries from public Common Vulnerabilities and Exposures (CVE) databases and related source code repositories. Consequently, all samples within the *Real-Vul* dataset are real and written in C/C++ as well. Our focus on projects with a higher number of vulnerabilities signifies the substantial effort invested in identifying vulnerabilities in these projects [47]. Our selection consists of the top 10 projects with the most vulnerabilities, creating a dataset that is at least two times larger than those used in previous studies [4], [54]. Remarkably, these ten projects encompass 73% of the vulnerable samples from the entire Big-Vul dataset, making *Real-Vul* a representative subset without its limitations.

Creation of vulnerable samples. To align with real-world scenarios, we adopt a time-based strategy for sample creation in the training and testing datasets. This approach simulates the process of training models on historical data and then identifying vulnerabilities over time. The time-based strategy is described below:

For generating vulnerable samples, we start extracting the dates on which the vulnerable functions were fixed using the vulnerability-fixing commit hashes available in the Big-Vul dataset.

To organize vulnerable samples, we order the functions based on their vulnerability-fixing dates (i.e., the dates of

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:03:49 UTC from IEEE Xplore. Restrictions apply.

TABLE I
PROJECTS INCLUDED IN *REAL-VUL* DATASET

Project	Description	Project Size (# of Functions)	# Vulnerable Functions	# Uncertain Functions
Chromium	Open source browser	153,057	3,137	149,920
FFmpeg	Video and audio processing	7,071	85	6,986
ImageMagic	Image manipulation tool	1,401	201	1,200
Jasper	Image encoding tool	315	86	229
Krb5	Computer network authentication protocol	3,151	106	3,045
Linux	Operating system	91,392	1,477	89,915
Openssl	Encryption tool suit	2,568	110	2,458
Php-src	Interpreter for php-scripting language	3,613	104	3,509
Qemu	System emulator	7,739	82	7,657
Tcpdump	Computer network analyzing tool	612	140	472

vulnerability-fixing commits). We then allocate the first 80% of the ordered vulnerable functions to the training dataset, while the remaining 20% form the test dataset. For example, in the case of the FFmpeg project, the training dataset includes samples fixed between August 3, 2013, and May 30, 2018. The testing set comprises samples fixed between June 28, 2018, and August 5, 2019. Note that these dates are the vulnerability-fixing commit dates in the FFmpeg project. The dates for all of the projects are available in the online Appendix. In total, our dataset consists of 5,528 vulnerable functions, with 4,418 functions assigned to the training dataset and 1,110 to the test dataset.

Creation of uncertain samples. To create uncertain samples, we clone the remote repository of each selected project and check out two snapshots, one for the training dataset and one for the testing dataset, using the most recent snapshots for each. For instance, in the FFmpeg project example, we clone the FFmpeg repository on May 31, 2018, to generate uncertain samples for the training dataset. This ensures that the uncertain samples are from a version later than the last vulnerability-fixing date in the training dataset (May 30, 2018). For the test dataset, we take the second snapshot on August 6, 2019. It is essential to maintain the constraint of chronological dates for training and testing snapshots. That said, other snapshots of the project could be taken as long as this constraint is upheld.

To obtain uncertain functions, we extract all functions from the source code files collected during snapshot cloning. We calculate the hash of these collected functions and the previously gathered vulnerable functions. Functions whose MD5 hashes do not match any of the MD5 hashes of the vulnerable functions are labeled as uncertain, ensuring label consistency in the *Real-Vul* dataset.

Upon completion, we have a total of 1,682,713 uncertain functions, with 769,464 allocated to the training dataset and 913,249 to the test dataset. The higher number of uncertain functions in the test dataset is due to their extraction from a later project version compared to the one used for the training dataset.

IV. STUDY DESIGN

In this section, we present the models we evaluate using *Real-Vul* dataset (Section IV-A). Then, we describe the research questions driving our investigation (Section IV-C), and the evaluation metrics (Section IV-B).

A. Employed Models

We choose four *state-of-the-art* deep learning-based models, namely LineVul [20], DeepWukong [7], ReVeal [4], and IVDetect [30] to conduct our experiments. Next, we briefly describe the architectural details of these models.

LineVul. LineVul [20] is a deep learning-based model built using CodeBERT [17]. We opt to include LineVul in our analysis as it is the state-of-the-art deep learning-based vulnerability detection model that utilizes a sequence-based approach for vulnerability detection. LineVul takes in a chunk as input and classifies it as a vulnerable or uncertain *chunk* utilizing the CodeBERT model. A chunk is a sequence of code tokens generated from source code programs. The LineVul has been trained on the Big-Vul dataset which has been curated using vulnerability-fixing commits from open-source projects.

DeepWukong. DeepWukong [7] is a leading vulnerability detection model utilizing a Graph Neural Network (GNN) [2] for in-depth code analysis. It converts code into Program Dependence Graphs (PDGs), then into XFGs (subgraphs), highlighting the data and control flow dependencies within. DeepWukong evaluates these XFGs to identify vulnerable code segments, employing the SARD dataset for training. This model has been included in the study due to its advanced graph-based approach, which sets a new benchmark for accurately detecting code vulnerabilities.

ReVeal. ReVeal [4] is a model that finds vulnerable code using graph neural networks. It has been chosen because it gathers data similarly to *Real-Vul*, focusing on unchanged functions from specific commits as non-vulnerable examples. ReVeal combines control flow, data flow, syntax trees, and dependency graphs into a Code Property Graph (CPG) for comprehensive

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:03:49 UTC from IEEE Xplore. Restrictions apply.

code analysis. It trains on a dataset curated from the Linux Debian Kernel and Chromium project.

IVDetect. IVDetect [30] is a tool that aims to provide precise interpretations of detected vulnerabilities. IVDetect incorporates representation learning and a graph-based interpretation model. It processes code by analyzing control flow, data flow, abstract syntax trees, and program dependency graphs, creating a unified structure for efficient vulnerability detection. IVDetect uses datasets created by prior studies [4], [16] comprising projects such as FFmpeg, Qemu, and Chromium.

B. Evaluation Metrics

The evaluation metric must accurately measure the models' performance on the task at hand. In this study, we use accuracy, precision, recall, F1, and AUC to measure the models' performance.

Accuracy quantifies the overall correctness of a vulnerability detection model, representing the proportion of true results (both true positives and true negatives) among the total number of samples that were evaluated. For example, if a model correctly identifies 80 out of 100 functions (vulnerable or not), its accuracy is 80%. High accuracy indicates effective identification of both vulnerable and uncertain samples with minimal errors.

Precision represents the fraction of vulnerabilities detected by the model that are truly vulnerabilities. For example, in the case of the DeepWukong model, it measures how many XFGs predicted as vulnerable are genuinely vulnerable XFGs. A low precision score indicates that the model is incorrectly classifying many uncertain samples as vulnerable, leading to a high number of false positives. Conversely, a high precision score implies that when the model detects a vulnerability, it is likely a real vulnerability.

Recall, on the other hand, denotes the fraction of actual vulnerabilities in the system that the model successfully detects. For the DeepWukong model, it measures how many true vulnerable XFGs are correctly identified as vulnerable by the model. A high recall score indicates that the model can detect most vulnerabilities correctly, resulting in a low number of false negatives.

The F1 score combines precision and recall, providing an overall assessment of how well these two measurements are balanced. By considering both precision and recall, the F1-score offers a more complete understanding of the model's effectiveness in vulnerability detection.

Area Under the Curve (AUC) evaluates a model's capability to distinguish between classes, like differentiating vulnerable from uncertain XFGs in the DeepWukong model. A high AUC score indicates that the model is proficient in accurately distinguishing between vulnerable and uncertain XFGs. Notably, in a balanced dataset, a random model would achieve an AUC of 0.50.

C. Research Questions

We introduce two research questions (RQs) and explain the motivation behind each one.

RQ$_1$: *How do the DeepWukong, LineVul, ReVeal, and IVDetect models perform in a realistic evaluation setting compared to the evaluation setting used in the original studies?*

DeepWukong, LineVul, ReVeal, and IVDetect are four state-of-the-art vulnerability detection models that have demonstrated promising results in identifying security vulnerabilities. However, we contend that the datasets used to evaluate these models do not accurately represent real-world usage, where the models would scan entire project source code files (see Section II for a comprehensive overview of dataset limitations). For instance, the DeepWukong model heavily relies on the SARD dataset for evaluation, which contains artificially generated samples lacking the complexities of real-world vulnerabilities. Similarly, the LineVul model used the Big-Vul dataset [16], which suffers from a scarcity of uncertain samples. Additionally, the ReVeal dataset also suffers from limited uncertain samples and significant "label-inconsistency." In this RQ, we evaluate the DeepWukong, LineVul, ReVeal, and IVDetect models using our *Real-Vul* test dataset, which provides a more realistic representation of vulnerability detection model performance in practical scenarios.

RQ$_2$: *How do the DeepWukong, LineVul, ReVeal, and IVDetect models perform in a realistic evaluation setting when trained using a similar realistic training dataset?*

In RQ$_1$, we utilize the *Real-Vul* dataset for model evaluation. However, we recognize that the training dataset employed to train these models might not accurately represent the same distribution as the evaluation dataset (i.e., *Real-Vul*). Doing so can lead to poor model performance as the training and testing datasets are from different distributions. To achieve optimal results, it is essential for both datasets to share the same distribution as the data encountered in practical usage. Consequently, in RQ$_2$, we investigate the impact of data distribution on model performance by training and testing the models using our *Real-Vul* dataset. This exploration will enable us to ascertain whether employing a training dataset representative of the same distribution as the evaluation dataset leads to enhanced model performance.

V. Preliminary Analysis

Our study aims to evaluate the performance of the DeepWukong, LineVul, and ReVeal models in a more realistic setting. As a first step towards our goal, we run the DeepWukong, LineVul, ReVeal, and IVDetect models on the SARD, the Big-Vul, the ReVeal, and the IVDetect datasets, respectively, and verify the findings reported in the original papers. By doing so, we aim to ensure that the results previously reported are reliable and can be replicated. Additionally, we use the results obtained from these experiments as our baseline models, which we compare to the performance of the models in our introduced realistic settings.

Approach. To evaluate the DeepWukong, LineVul, ReVeal, and IVDetect models on the dataset used in their studies, we first download the SARD dataset,[2] the Big-Vul dataset,[3] the

[2]https://github.com/jumormt/DeepWukong
[3]https://github.com/awsm-research/LineVul

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:03:49 UTC from IEEE Xplore. Restrictions apply.

TABLE II
REPLICATION RESULTS OF DEEPWUKONG, LINEVUL, REVEAL, AND IVDETECT MODELS

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)	AUC (%)
DeepWukong	98	87	98	93	88
LineVul	96	96	84	90	85
ReVeal	81	29	59	38	78
IVDetect	85	39	63	24	83

ReVeal dataset,[4] used in DeepWukong [7], LineVul [20], ReVeal [4], and IVDetect [30] studies, respectively. We create the DeepWukong model inputs (XFGs), the LineVul model inputs (chunks), the ReVeal model inputs (CPG), and the IVDetect models input PDG using the SARD dataset, the Big-Vul dataset, the ReVeal dataset, and the IVDetect dataset samples, respectively. The generated XFG, chunk, CPG, and PDG datasets are split into training and testing datasets where 80% of the samples belong to the training dataset and the remaining 20% samples belong to the testing dataset. Following the authors' original experiments, we train the DeepWukong model for 50 epochs and the LineVul model for ten epochs. For the ReVeal and IVDetect model, we follow the original implementation of ReVeal and IVDetect. The maximum epoch is set to 100 and 50 for ReVeal and IVDetect, respectively. We stop the training procedure if the F1-score does not increase for five consecutive epochs. Finally, We evaluate the models using the testing datasets.

Results. Table II shows the replication results of the DeepWukong, LineVul, ReVeal and IVDetect models. From the figure, we observe that the DeepWukong model achieves 98%, 87%, 98%, 93%, and 88% for accuracy, precision, recall, F1-score, and AUC, respectively. The accuracy and F1-score for the DeepWukong model differ by +1 and −2 percentage points, respectively, from the results reported by the DeepWukong paper.

Our results for the LineVul model follow a similar trend. The accuracy, precision, recall, F1-score, and AUC for the LineVul model are 96%, 96%, 84%, 90%, and 85% respectively. Comparing with the results reported in the LineVul paper, we find that the precision, recall, and F1-score differ by −1, −2, and −1 percentage points, respectively.

The ReVeal model demonstrates an accuracy of 81%, with precision, recall, F1-score, and AUC values of 29%, 59%, 38%, and 78%, respectively. Upon comparing these results with the findings reported in the ReVeal paper, we note a difference of −6, −3, and −7 percentage points in precision, recall, and F1-score, respectively.

The IVDetect model demonstrates an accuracy of 85%, with precision, recall, F1-score, and AUC values of 39%, 63%, 24%, and 83% respectively. Upon comparing these results with the findings reported in the IVDetect paper, we note a difference of −2, −2, and −24 percentage points in precision, recall, and F1-score, respectively.

Overall, we observe that the differences in the metrics are negligible. The little difference in the metrics can be because of the presence of different samples in the training/testing datasets. When a dataset is split into a training set and a testing set, the samples in the two sets are chosen randomly. This means that the samples in the training set and the testing set will be different each time we split the dataset. As a result, model performance on the testing set may vary from one split to another. The model performance on the testing set may depend on the specific samples that are included in the testing set. For example, if the testing set contains a particularly difficult or easy sample, this can affect the overall performance. These results serve as baseline models for comparison with the performance of the models tested under realistic settings in the RQs.

The effectiveness of these models in vulnerability detection is influenced by the degree to which the embeddings of the vulnerable and uncertain classes are distinct and separable. That is, the greater the distinction and separability of the embedding, the more straightforward it is for the model to differentiate between the two classes. To explore this aspect, we conduct an experiment that involves visualizing the model's capability to distinctly segregate samples belonging to the vulnerable and uncertain classes.

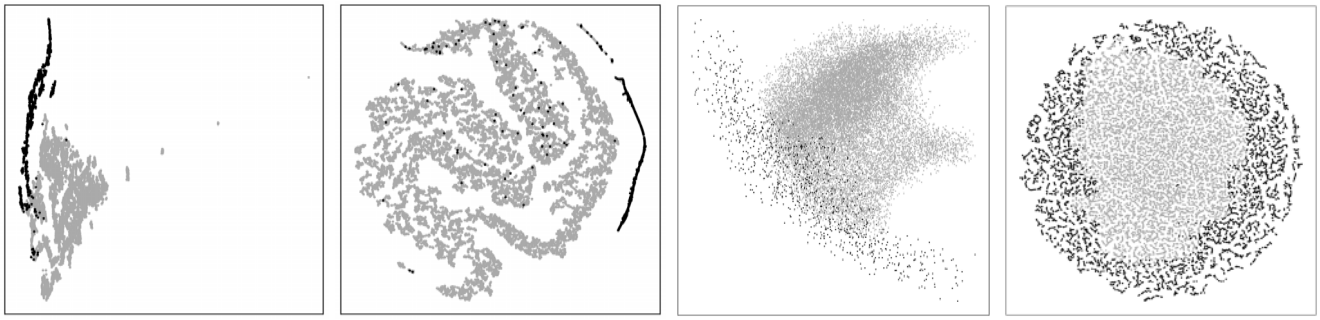
We employ t-distributed Stochastic Neighbor Embedding (t-SNE) [46] to visualize the embeddings produced by the DeepWukong, LineVul, ReVeal, and IVDetect models. T-SNE is a machine learning algorithm for visualizing high-dimensional data in a low-dimensional (typically two or three) space. It calculates similarities between data points, converting them into probabilities and minimizing their Kullback-Leibler divergence [26] to preserve local structures. This process effectively clusters similar data, which makes t-SNE valuable for identifying patterns and relationships in multi-dimensional data.

We extract the embedding from the DeepWukong, LineVul, ReVeal, and IVDetect models used in this RQ. The *[CLS]* embedding vectors generated by the CodeBERT model represent the embedding for the LineVul model, while the final fixed vector produced by the Graph Neural Network and the representation-learning model represents the embedding for the DeepWukong, ReVeal, IVDetect model, respectively. It is worth noting that the embeddings are generated for the testing datasets.

We create four scatter plots in total, each displaying the reduced embedding along with their corresponding labels (vulnerable or uncertain). The scatter plots are presented in Fig. 2. We observe that all the models we created, except ReVeal and IVDetect, exhibit clear separation between vulnerable and uncertain samples. The inability to clear separation may explain the comparatively lower performance of the ReVeal and IVDetect model.

[4]https://drive.google.com/drive/folders/1KuIYgFcvWUXheDhT–cBALsfy1I4utOyReVeal

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:03:49 UTC from IEEE Xplore. Restrictions apply.



(a) DeepWukong. (b) LineVul. (c) ReVeal. (d) IVDetect.

Fig. 2. Scatter plots showing the class separation between vulnerable and uncertain samples in the original datasets of the approaches. ● denotes vulnerable samples and ● denotes uncertain samples.

TABLE III
RESULTS OF THE DEEPWUKONG, LINEVUL, REVEAL, AND IVDETECT MODELS WHEN EVALUATED USING THE *REAL-VUL* TESTING DATASET

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)	AUC (%)
DeepWukong	91	1	87	2	62
LineVul	89	1	90	2	58
ReVeal	89	10	80	17	61
IVDetect	85	2	84	2	59

VI. RESULTS

In this section, we present our experimental results with respect to each RQ.

RQ$_1$: *How do the DeepWukong, LineVul, ReVeal, and IVDetect models perform in a realistic evaluation setting compared to the evaluation setting used in the original studies?*

Approach. In this RQ, we evaluate the DeepWukong, the LineVul, the ReVeal, and the IVDetect models using the *Real-Vul* testing dataset described in Section III. We use the LineVul, ReVeal, and the IVDetect model trained in Section V for the evaluation, i.e., we train using the same method as the original LineVul, ReVeal, and IVDetect studies, respectively, while using *Real-Vul* testing dataset for evaluation. Note that LineVul, ReVeal, and IVDetect models are trained using real-world vulnerability datasets. Since the DeepWukong model in Section V is trained using SARD (an artificially created vulnerability dataset), we train a new DeepWukong model using the Big-Vul dataset for evaluation to simulate a fair comparison with the other two models. First, we generate the DeepWukong model inputs (XFGs) using the Big-Vul dataset. We obtain a total of 28,294 vulnerable XFGs and 639,047 uncertain XFGs. Similar to preliminary analysis, we train the DeepWukong model for 50 epochs using the generated XFGs.

To evaluate the trained models (DeepWukong, LineVul, ReVeal, and IVDetect) using the *Real-Vul* testing dataset, we first generate the model inputs (XFG, chunk, CPG, and PDG) for the *Real-Vul* testing dataset. These samples are then evaluated using the respective trained models.

Results. Table III shows the results of the performance of the DeepWukong, LineVul, ReVeal, and IVDetect models evaluated using the *Real-Vul* testing dataset. The DeepWukong model obtains an accuracy, precision, recall, F1-score, and AUC of 91%, 1%, 87%, 2%, and 62%, respectively. When comparing these results to those obtained in Section V (i.e., compared to the results of the DeepWukong model trained and tested using the SARD dataset), we observe a decrease of 7, 86, 11, and 91 percentage points in accuracy, precision, recall, and F1-score, respectively.

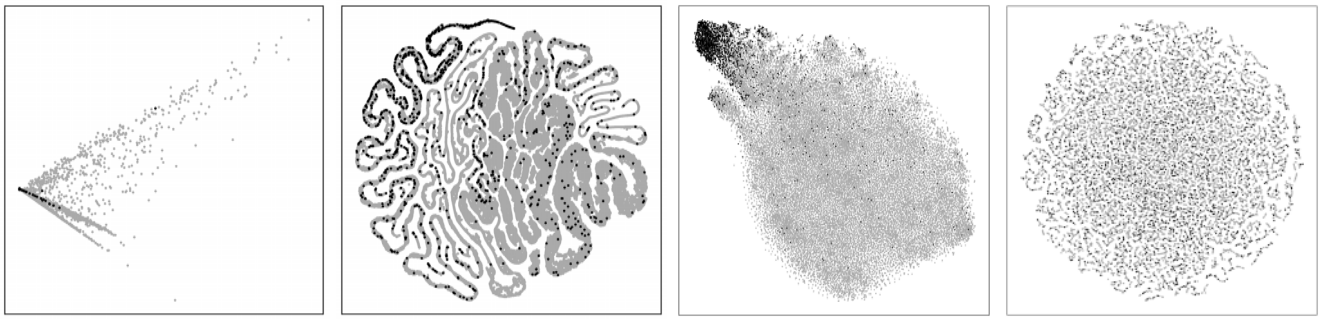
The LineVul model obtains an accuracy, precision, recall, F1-score, and AUC of 89%, 1%, 90%, 2%, and 58%, respectively. In comparison to the results obtained in Section V (i.e., compared to the results of the LineVul model trained and tested using the Big-Vul dataset), we observe a decrease of 7, 95, 88, and 27 percentage points in accuracy, precision, F1-score, and AUC, respectively. However, we also observe an increase of 6 percentage points in the recall.

The ReVeal model demonstrates an 89% accuracy rate, with precision, recall, and F1-score values of 10%, 80%, and 17%, respectively. When comparing these results to the results from Section V where the ReVeal model was trained and tested using the ReVeal dataset, we observe a decrease of 19, 21, and 17 percentage points in precision, F1-score, and AUC, respectively, but a 21 percentage points improvement in recall.

The IVDetect model exhibits an accuracy of 85%, along with precision, recall, and F1-score metrics of 2%, 84%, and 2% respectively. In comparison to the outcomes from Section V, where the IVDetect model underwent training and testing using the IVDetect dataset, a notable reduction of 37 percentage points in precision, 22 percentage points in the F1-score, and 24 percentage points in AUC can be observed; however, an enhancement of 21 percentage points in recall is also observed.

Overall, our findings reveal a substantial decrease in precision for all the models. This suggests that the predictions made by these models contain a considerable number of false positives, which can have a huge impact on the effectiveness

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:03:49 UTC from IEEE Xplore. Restrictions apply.



(a) DeepWukong with *Real-Vul*. (b) LineVul with *Real-Vul*. (c) ReVeal with *Real-Vul*. (d) IVDetect with *Real-Vul*.

Fig. 3. Scatter plots showing the class separation between the vulnerable and uncertain samples. ● denotes vulnerable samples and ● denotes uncertain samples.

of vulnerability detection. Specifically, the DeepWukong model generated 439,494 false positives, while the LineVul, ReVeal, and IVDetect models produced 114,629, 320,128, and 512,284 false positives, respectively. Such a large number of false positives may have a large impact on the usability of these models in practice.

To understand the models' performance in terms of class separation, we follow the same approach used in Section V, and plot embeddings of the models in two-dimensional space. It is worth noting that the embeddings are generated for the test split of *Real-Vul* dataset. Fig. 3 presents the scatter plots of model embeddings. For all the models, we observe a substantial overlap between vulnerable and uncertain samples of the *Real-Vul* dataset, which indicates that the models do not distinguish between the classes clearly. This lack of class separation explains the high number of false positives produced by the DeepWukong, LineVul, ReVeal, and IVDetect models.

Overall, our findings underscore the importance of evaluating vulnerability detection models using datasets that reflect realistic settings. In this regard, our results demonstrate the critical role played by the *Real-Vul* dataset in understanding the true capabilities of such models. By utilizing this dataset, we gain a more accurate understanding of the performance of vulnerability detection models and can identify areas that require improvement.

ANSWER: Existing deep learning-based vulnerability detection models, such as DeepWukong, LineVul, ReVeal, and IVDetectproduce a high number of false positives when tested using datasets that represent more accurate real-world testing settings.

RQ$_2$: *How do the DeepWukong, LineVul, ReVeal, and IVDetect models perform in a realistic evaluation setting when trained using a similar realistic training dataset?*

Approach. In RQ$_1$, we use the *Real-Vul* dataset for evaluation purposes, i.e., we evaluate the effectiveness of our models using *Real-Vul*. However, in this RQ, we use the *Real-Vul* dataset for both model training and evaluation. The training dataset is imbalanced (higher uncertain samples). Hence, we also investigate the impact of the imbalanced dataset on model performance by training additional models on a balanced dataset. To create a balanced dataset for training, we randomly select uncertain samples equal to the number of vulnerable samples.

TABLE IV
RESULTS OF THE DEEPWUKONG, LINEVUL, REVEAL, AND IVDETECT MODELS WHEN TRAINED USING THE IMBALANCED DATASET (*REAL-VUL* DATASET)

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)	AUC (%)
DeepWukong	99	0	0	0	50
LineVul	99	0	0	0	50
ReVeal	95	2	10	3	50.1
IVDetect	81	2	1	0	49.6

Overall, we train a total of eight models (one imbalanced-trained and one balanced-trained model for DeepWukong, LineVul, ReVeal, and IVDetect techniques) using the same training parameters used in Section V. We evaluate the trained DeepWukong, LineVul, ReVeal, and IVDetect models using the same XFG, chunk, CPG, and PDG test dataset used in RQ$_1$ (i.e., *Real-Vul* test dataset).

Results. Table IV shows the results of the DeepWukong, LineVul, ReVeal, and IVDetect models that are trained using imbalanced datasets. The DeepWukong and LineVul models both exhibit an accuracy of 99%, with precision, recall, F1-scores at 0%, and AUC at 50%. In comparison, the ReVeal model exhibits 95% accuracy, 2% precision, 10% recall, 3% F1-score, and 50% AUC. Similarly, the IVDetect model demonstrates an 81% accuracy, with precision, recall, F1-score, and AUC recorded at 2%, 1%, 0%, and 49.6%, respectively. The accuracy is 99% for both the DeepWukong and LineVul models, while the precision, recall, F1-scores, and AUC are 0% and 50%, respectively. For the ReVeal model, the accuracy, precision, recall, F1-score, and AUC are 95%, 2%, 10%, 3%, and 50%, respectively. A common trend across the models is that the

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:03:49 UTC from IEEE Xplore. Restrictions apply.

TABLE V
RESULTS OF THE DEEPWUKONG, LINEVUL, REVEAL, AND IVDETECT MODELS WHEN TRAINED USING THE BALANCED DATASET (*REAL-VUL* DATASET)

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)	AUC (%)
DeepWukong	89	1	53	2	51.4
LineVul	99	11	99	20	51.4
ReVeal	91	31	45	36	51.3
IVDetect	83	8	32	6	52.3

precision, recall, and F1-scores are substantially reduced with respect to the results in Section V and RQ_1.

The high accuracy scores of the models are due to the imbalanced nature of the datasets, which have a larger number of uncertain samples. This imbalance leads models to often predict samples as uncertain, boosting accuracy but reducing recall, precision, and F1-scores. The models have difficulty learning from the fewer vulnerable samples because they focus on minimizing a loss function, which discourages wrong predictions. Consequently, they tend to classify samples as uncertain to reduce errors despite inaccuracies. However, the ReVeal model performs better as it uses the SMOTE [5] algorithm to balance the classes by oversampling the minority (vulnerable samples).

Table V shows the performance of the DeepWukong, LineVul, ReVeal, and IVDetect models that are trained using balanced datasets (XFGs, chunks, CPGs, and PDGs). The results indicate that the DeepWukong model trained on the balanced dataset performs with 89% accuracy, 1% precision, 53% recall, 2% F1-score, and 51.4% AUC, which is down by 9, 86, 45, 91, and 36 percentage points, respectively, compared to the DeepWukong model evaluated in Section V. The LineVul model trained on the balanced dataset performs with 99% accuracy, 11% precision, 99% recall, 20% F1-score, and 51.4% AUC. The precision, F1-scores, and AUC are down by 85, 70, and 33.6 percentage points, respectively, compared to the LineVul model evaluated in Section V. When trained on the balanced dataset, the ReVeal model showcases a performance of 91% accuracy, 31% precision, 45% recall, 36% F1-score, and 51.3% AUC. In comparison to the ReVeal model evaluated in Section V, the recall, F1-scores, and AUC experience a decline of 14, 2, and 26 percentage points, respectively. The IVDetect model trained on the balanced dataset performs with 83% accuracy, 8% precision, 32% recall, 6% F1-score, and 52.3% AUC. The precision and recall are reduced by 31 percentage points, whereas accuracy, F1-score, and AUC are reduced by 2, 18, and 30 percentage points, respectively, compared to the IVDetect model of Section V.

Overall, the results suggest that these models still produce a high number of false positives, even when trained using a dataset that is similar to the realistic evaluation dataset.

When we compare how the models did compare to earlier evaluation (RQ_1) with the original dataset, we found that the recall scores for the DeepWukong, ReVeal, and IVDetect models dropped by 34, 35, and 52 percentage points, respectively. The differences in how well the DeepWukong, LineVul, ReVeal, and IVDetect models performed could be because of the kind of input they use (like XFG, chunks, CPGs, and PDGs) and model architecture (GNN, CodeBERT). Prior research [48] shows that models using graphs usually perform a bit better than those using text, which helps us understand why ReVeal (using graphs) performed better than LineVul (using text).

We assess the models when trained on *Real-Vul* and tested on their corresponding original datasets (such as SARD, ReVeal, BigVul, and IVDetect) to cross-check the impact of realistic settings. Our experiments indicate a substantial improvement in performance, with AUC improvements ranging from 8 to 32 percentage points compared to the model trained and tested solely on the *Real-Vul* dataset. Specifically, on the original test dataset, the DeepWukong model achieved an accuracy of 88%, precision of 81%, recall of 89%, F1-score of 84%, and an AUC of 81%, observing improvements over its *Real-Vul* dataset performance in every metric except accuracy, which decreased by one percentage point. The LineVul model achieved an accuracy of 80%, precision of 84%, recall of 69%, F1-score of 75%, and AUC of 79.9%, showing declines in accuracy and recall by 19 and 30 percentage points, respectively, but gains in precision, F1-score, and AUC. The ReVeal model experienced a 5 percentage point decrease in accuracy while seeing improvements of 4, 6, 5, and 8 percentage points in precision, recall, F1-score, and AUC, respectively. The IVDetect model achieved an accuracy of 78%, precision of 28%, recall of 48%, F1-score of 35%, and AUC of 78.3%, with a decrease in accuracy by five percentage points but increases in all other metrics. Overall, the improved performance suggests that the prior datasets might not accurately represent realistic settings, thus overestimating model performance.

The study found that using a balanced dataset for training significantly improves model performance in detecting vulnerabilities. Specifically, the DeepWukong model showed improvements in precision, recall, and F1-scores by 1, 53, and 2 percentage points, respectively, after training on a balanced vs. an imbalanced dataset. This improvement trend was also observed in the LineVul, ReVeal, and IVDetect models. However, despite these gains, the issue of generating false positives remains a challenge for all models.

ANSWER: Despite training the DeepWukong, LineVul, ReVeal, and IVDetectmodels with a realistic training dataset that closely resembles the realistic test dataset, we observe that these models still generate a high number of false positives (low precision).

VII. DISCUSSION

In this section, we delve into the false positive predictions and further examine the performance of the models under review by evaluating their effectiveness across different types of vulnerabilities and levels of severity.

Analysis of false positives. To understand why the studied models produce false positives, we have conducted new analyses from three perspectives.

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:03:49 UTC from IEEE Xplore. Restrictions apply.

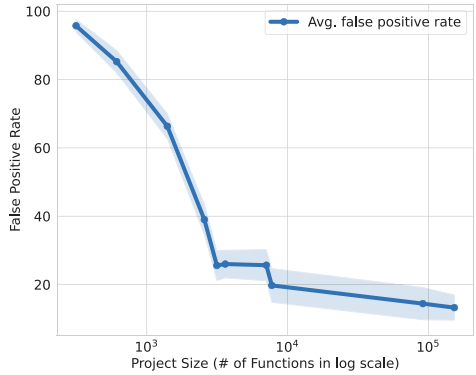


Fig. 4. Project Size vs. False Positive Rate for the analyzed projects.

TABLE VI
RESULTS OF THE DEEPWUKONG, LINEVUL, REVEAL, AND IVDETECT MODELS WHEN TRAINED USING THE AUGMENTED DATA. (*VALUES IN PARENTHESES SHOW THE DIFFERENCE FROM TRAINING ON A NON-AUGMENTED BALANCED DATASET*)

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)	AUC (%)
DeepWukong	91 (↑ 2)	4 (↑ 3)	48 (↓ 4)	7 (↑ 5)	58 (↑ 6.6)
LineVul	98 (↓ 1)	37 (↑ 26)	65 (↓ 34)	46 (↑ 26)	81.8 (↑ 30.4)
ReVeal	92 (↓ 1)	33 (↑ 2)	47 (↑ 2)	38 (↑ 2)	57 (↑ 5.6)
IVDetect	78 (↓ 5)	10 (↑ 2)	35 (↑ 3)	16 (↑ 10)	53 (↑ 0.7)

1) **Size of Project.** The motivation for considering project size as a critical factor arises from the notion that larger projects offer a more extensive dataset for model learning. This allows models to better comprehend and identify complex vulnerability patterns, potentially leading to a reduction in false positive rates. In Fig. 4, we illustrate the influence of project size on false positive rates where we observe a downward trend, i.e., as the size of the project increases, the false positive rate correspondingly decreases. Comparing the smallest project, Jasper, consisting of 183 training and 132 testing samples, with larger projects such as Chrome and Linux, which encompassed 44,303 and 71,706 training samples and 81,351 and 47,181 testing samples, respectively. The findings supported our hypothesis: larger projects exhibited substantially lower false positive rates (85%-87% for Chrome and 79%-85% for Linux).
2) **Code complexity analysis.** Machine learning (ML) models often struggle to comprehend highly complex code [50], which can affect the performance of vulnerability detection tools. To investigate this, we measured the cyclomatic complexity across our test dataset and conducted Mann-Whitney U tests to compare complexities across different categories: true positive vs. false positive ($p = 0.043$) and true negative vs. false negative ($p = 0.014$). Our comparison suggests that increased complexity leads to model confusion, resulting in higher false positive rates.
3) **Overfitting assessment.** ML models may assign excessive importance to specific features, a phenomenon referred to as "overfitting" [35]. This tendency can detrimentally impact model performance. For a detailed examination of "overfitting," we employed LIME [43], an explainability tool, to pinpoint the features influencing the models' decisions. An example of the LIME explanation is available in the online Appendix 1. Note that our analysis encountered challenges with three of our models DeepWukong, ReVeal, and IVDetect —that rely on Joern[5] to convert source code into abstract graphs. This process introduced complexities in tracing back to the original code for explainability purposes. Consequently, we restricted our analysis to false positives identified solely by the LineVul model. To confirm our findings, we conducted hypothesis testing. First, we create a set of positive tokens from the LIME output (the model predicts positive). Second, we calculate the ratio of positive tokens available in the test dataset samples. Finally, we apply the Mann-Whitney U test with $\alpha = 0.05$ to compare the ratio in false positive and true positive samples but find no significant statistical difference ($p = 0.13$).

[5]https://github.com/joernio/joern

To address this challenge, we propose augmenting the dataset with dead code, a technique that enhances dataset diversity more effectively than simple resampling by creating varied minority class examples, avoiding the reinforcement of misleading patterns [42]. Dead code, which does not influence program output, introduces variations in textual data through syntactic transformations while preserving the original meaning. By broadening the hypothesis space for deep learning models, augmentation serves as implicit regularization, reducing overfitting to specific tokens and promoting model generalization [41].

Influenced by [36], we augmented the training dataset with dead code to enrich the diversity and mitigate the overemphasis on specific tokens. We employed eleven distinct augmentation strategies, elaborated in the online appendix. We particularly focused on augmenting vulnerable samples to balance them with uncertain (non-vulnerable) samples, thereby enhancing dataset diversity.

We train the four models in our study using the augmented dataset and evaluate them against the test data. Table VI provides an overview of the performance metrics. Compared to models that we train using balanced datasets, the AUC substantially improves for all models. Specifically, we find that the text-based model "LineVul" improved more substantially in terms of F1 and AUC than the graph-based models (i.e., DeepWukong, LineVul, ReVeal, and IVDetect). Conversely, we find that the recall of the LineVul model drops by 34 percentage points. The previous model labeled a large proportion of samples as vulnerable and, consequently, achieved an almost perfect recall (at the cost of low precision). LineVul 's improvement in F1 suggests that the improvement in precision (26 percentage points) outweighs the cost of the recall. We conclude that augmentation can mitigate false positives and improve the practical applicability of vulnerability detection models.

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:03:49 UTC from IEEE Xplore. Restrictions apply.

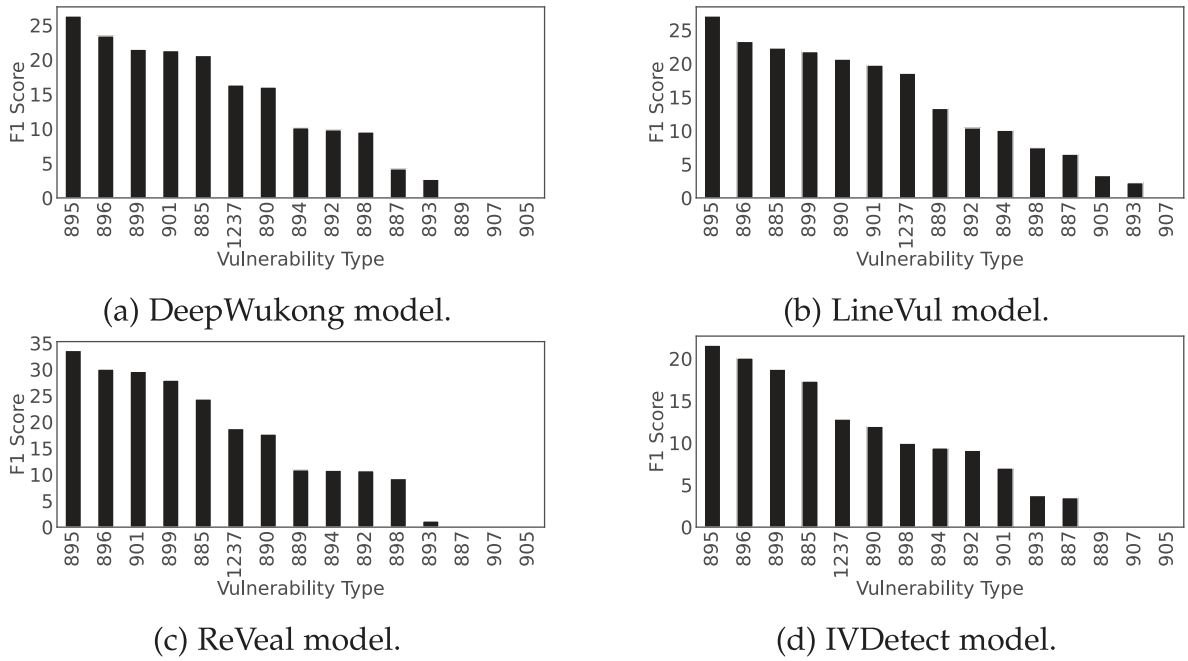


Fig. 5. Performance of the models per SFP cluster.

Note that while text-based models benefit from the variability introduced by dead code, graph-based models may require augmentations that induce structural or data flow changes. Hence, we encourage future work to enhance the effectiveness of dead code injection for graph-based models. The disparity in the impact of dead code injection between text-based and graph-based models stems from their differing code processing approaches. Text-based models may experience enhanced performance due to the variability introduced by dead code, perceiving it as a regularization feature [49]. Conversely, graph-based models, which leverage PDGs or DFGs, may regard the dead code as noise, as it does not alter the fundamental program structure or data flow, leading to marginal performance improvements.

Performance per vulnerability type. In our dataset, each vulnerability sample is associated with a specific Common Weakness Enumeration (CWE). The security community utilized CWEs to group vulnerability types into clusters based on their common characteristics. These clusters are collectively known as the Software Fault Pattern (SFP) [10], and they consist of sets of similar CWEs. To assess the performance of the models in detecting vulnerabilities belonging to different clusters, we first calculate the F1-score (using the model trained in RQ_2) for each CWE category. Then, we group these CWEs based on the Software Fault Pattern (SFP) clusters.

Fig. 5 presents the performance of the studied models in terms of F1-scores across different clusters. Overall, we observe that the performance varies across different clusters, with values up to 26.34% for DeepWukong, 27.10% for LineVul, 33.61% for ReVeal, and 21.60% for IVDetect. For instance, the models perform exceptionally well in detecting vulnerabilities that belong to clusters 895 or 896, with F1-scores of 33.6% and 23.31%, respectively. Cluster 895 includes CWEs associated with information leaks, such as returning a private data structure from a public method (CWE 495) or storing a password in plaintext (CWE 256). On the other hand, cluster 896 comprises CWEs related to tainted input, such as code injection (CWE 94) and improper input validation (CWE 20).

However, the models encounter more difficulties in detecting vulnerabilities belonging to clusters 893, 905, or 907. Cluster 893 contains CWEs related to path traversal (e.g., absolute path traversal CWE 36). Cluster 905 groups vulnerabilities related to predictability-related CWEs (e.g., predictable value range from previous values CWE 343), and cluster 907 consists of miscellaneous CWEs (e.g., client-side enforcement of server-side security CWE 602).

The models performed better at detecting vulnerabilities about information leaks or tainted inputs (clusters 895 and 896), possibly because these issues need less context to detect, with all the needed details present in the code itself. However, identifying vulnerabilities like path traversal (cluster 893) demands understanding the file system's structure, and cluster 905 (predictability issues) might need previous observations. Notably, no model could detect any vulnerabilities in cluster 907, which includes complex issues tied to the compiler, architecture, and design. Vulnerabilities may arise from the interactions between components, which may not be adequately represented by function-level vulnerability datasets. Our findings show that the performance of the model is lower on CWEs related to system-level vulnerabilities (e.g., CWE-893: Improper Path Traversal), suggesting that these models struggle to detect system-level vulnerabilities effectively.

However, it's important to consider that our observation might be influenced by data distribution bias. Typically, machine learning models perform better in specific categories

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:03:49 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
PERFORMANCE OF THE MODELS PER SEVERITY

Model	Low (%)	Medium (%)	High (%)
DeepWukong	18.26	20.56	9.7
LineVul	20.11	21.28	10.89
ReVeal	21.66	26.57	10.07
IVDetect	15.08	17.46	9.06

TABLE VIII
RESULTS OF THE CODELLAMA AND MIXTRAL MODELS WHEN TRAINED USING THE AUGMENTED DATA

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)	AUC (%)
CodeLlama	80	34	54	42	67
Mixtral	83	38	59	46	71

if they have sufficient data from those categories. To verify this assumption, we count the number of samples for each cluster in our dataset and identify the three most and least frequent clusters. We find that clusters 890 (32%), 896 (25%), and 892 (14%) are the most frequent in our dataset, while clusters 905 (0.048%), 889 (0.024%), and 907(0.003%) are the least frequent.

From Fig. 5, we can observe that the models achieve lower performance for the cluster with the highest frequency compared to other clusters with lower frequency. For example, the models did not achieve their highest F1-score when samples belonged to the most frequent cluster in the dataset (cluster 890). Conversely, the models achieve their lowest F1-scores for cluster 907, which is the least frequent sample in our dataset. This indicates that while other factors, such as architecture and input type, influence model performance in detecting vulnerabilities, the distribution of samples also plays a crucial role in making the model performance robust across different SFP clusters (CWE types). *The analysis indicates the inconsistent performance of vulnerability detection models across different CWE clusters, highlighting the influence of data distribution and contextual complexity. Future research in vulnerability detection should concentrate on enhancing data strategies and model architecture to address contextual complexity, as well as incorporating system-level vulnerabilities to improve the overall coverage of vulnerability detection models.*

Performance per vulnerability severity. In our investigation, we extend our analysis to consider the severity of vulnerabilities. In our dataset, each vulnerability is categorized into one of three severity types: high, medium, or low. To evaluate the models based on severity, we follow the same approach used for evaluating them per vulnerability type.

Table VII presents the performance of the models based on severity. The figure illustrates that all four models achieve relatively higher F1-scores when the severity of a vulnerability is medium, with a slightly lower F1-score for low-severity vulnerabilities. These findings raise concerns about vulnerability detection, as the models demonstrate weaker capabilities in detecting high-severity vulnerabilities.

It is essential to note that this observation may be influenced by the severity distribution within our *Real-Vul* dataset. Further analysis revealed that 60.5% of vulnerabilities in the dataset are assigned medium severity, while only 33.9% are assigned low severity, and 5% are assigned high severity. This distribution of severity levels could be contributing to model performances across different severity types. *The investigation reveals models are more effective at detecting medium-severity vulnerabilities, with decreased accuracy for high-severity ones. This underscores the need for future research to focus on enhancing the detection of high-severity vulnerabilities, considering the potential impact of severity distribution in training datasets.*

Efficacy of Large Language Models. Recent advancements in generative AI have led to the development of specialized Large Language Models (LLMs) for natural language understanding and code interpretation, such as CodeLlama [14] and Mixtral [13]. CodeLlama, a 7 B parameter model, excels in parsing and generating code across multiple programming languages, trained on a vast array of sources, including open-source projects and technical documentation, showcasing capabilities in code completion, bug detection, and retrieval. Mixtral [13] (7 B parameter) is a Sparse Mixture-of-Experts (SMoE) network. Its architecture can be divided into three parts: an input embedding layer, several decoder blocks, and a language model decoding head. The model is trained using supervised fine-tuning on an instruction dataset and Direct Preference Optimization (DPO) on a paired feedback dataset. The pre-training involves data sourced from the open web. In terms of performance, Mixtral has shown impressive results in various benchmarks, surpassing Llama 2 70B and equating or outperforming Open AI GPT-3.5 in most standard tests despite not being specifically tailored for code tasks.

We fine-tuned these models using parameter-efficient fine-tuning [11] to detect vulnerabilities in C++ functions, using the *Real-Vul* dataset for training and a separate test dataset for evaluation. The results, detailed in Table VIII, indicate that these LLMs outperform other tools like DeepWukong, ReVeal, and IVDetect in terms of AUC. However, when compared to LineVul, CodeLlama and Mixtral's AUC lags by 14.8 and 10.8 percentage points, respectively.

The lower performance of the LLMs in comparison to LineVul may stem from multiple reasons, such as data efficiency, model generalizability, and the trade-offs inherent in model complexity. LLMs, with their extensive parameter count, may not always generalize well to specific tasks outside their training scope, leading to suboptimal performance. In contrast, models like LineVul, through targeted training and optimization, can achieve better alignment with specific tasks. A similar observation has also been observed in another task where a RoBERTa model outperforms Llama 2 [15] and the Mixtral model [25]. *The study shows that specialized tools (LineVul) outperform general large language models (LLMs) in vulnerability detection, highlighting the need for future work to improve LLMs' specificity and efficiency in vulnerability detection tasks.*

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:03:49 UTC from IEEE Xplore. Restrictions apply.

VIII. Related Work

In this section, we discuss the related works and reflect on how they compare with ours.

Vulnerability detection datasets. Grahn et al. [21] found that some of the vulnerability detection datasets [16], [44] were not very useful for training models. To address this, they proposed a new dataset called *Wild C*, comprising 10.3 million C/C++ files from multiple open-source projects. However, the drawback of *Wild C* is that it lacks labels for each file, making it unsuitable for building vulnerability detection classification models. In contrast, the *Real-Vul* dataset offers the complete source code of projects along with labeled samples as vulnerable or uncertain, making it more suitable for training vulnerability detection models.

The Big-Vul dataset [16] is a collection of C/C++ functions from 348 open-source GitHub projects, used in vulnerability detection studies. However, it lacks representation of the entire codebase since it only includes functions from vulnerability-fixing commits. To address this, we introduce the *Real-Vul* dataset, which includes all source code from the top ten real-world projects by vulnerability counts in Big-Vul. This provides a more comprehensive and realistic dataset for training and evaluating vulnerability detection models.

Studies on vulnerability detection techniques. Several studies explored the effectiveness of traditional machine learning techniques [37], [51], [52]. Neuhaus et al. [37] investigated the prevalence of software vulnerabilities in Red Hat packages using Support Vector Machines (SVM) [23]. The study analyzed the defect data from over 3,241 Red Hat packages and evaluated the effectiveness of SVM in identifying vulnerabilities in software packages. Zheng et al. [52] examined the effectiveness of different machine learning techniques, like Decision Tree [40], Random Forests [1], k-nearest neighbors (KNN) [39], and SVM, in detecting software vulnerabilities. The results of the study provided insights into the strengths and weaknesses of different machine learning techniques for vulnerability detection. Yan et al. [51] employed a combination of static analysis, machine learning, and typestate modeling techniques for static detection of use-after-free vulnerabilities in software. Lomio et al. [34] investigated whether machine learning algorithms like SVM, KNN, Decision Tree, and Boosting algorithms [6], [18], [19] could improve the performance of Just-in-Time software vulnerability detection utilizing various software metrics (process metrics, product metrics, and text metrics).

Other studies focused on exploring the potential of various deep learning methods to detect vulnerabilities [7], [8], [20], [29], [31], [32], [54]. For example, Convolutional Neural Networks (CNN) [28] have been used to forecast software defects and locate defective source code [29]. Li et al. [31] made multiple kinds of deep neural networks such as CNN, LSTM [24], and GRU [9] to detect vulnerabilities. Vuldeepecker [32] detects resource management issues and buffer overflows by training an LSTM model with code embedding and data-flow information of a program. VGDetector [8] uses a control flow graph and a graph convolutional network [27] to detect control-flow vulnerabilities. Zhou et al. [54] pinpointed bugs at the method level using Graph Neural Networks (GNN) and program dependence graph.

The aforementioned work used either synthetic datasets (e.g., SARD), datasets created using oracles like static analysis tools (e.g., [45], [53]), or real-world datasets that do not accurately reflect the realistic settings (e.g., [4], [16], [54]). This motivated us to create a new dataset that tackles the lack of realistic evaluation settings in the existing techniques. We create *Real-Vul* dataset, which is more comprehensive than the existing vulnerability detection datasets.

Chakraborty et al. [4] introduced the ReVeal dataset, similar to our study, to highlight limitations in existing deep learning-based vulnerability detection models. However, they have considered only unchanged functions of vulnerability fixing commits as the uncertain sample. Additionally, the dataset suffers from label inconsistency. Fu et al. [20] proposed LineVul, a CodeBERT-based model [17], for vulnerability detection. Our study differs from the prior work since we propose a realistic dataset *Real-Vul* which is free from the limitations of prior studies. Additionally, we evaluate four *state-of-the-art* techniques (i.e., [4], [7], [20]) on *Real-Vul*, providing the empirical evidence that the machine learning models exhibit limited performance when assessed in real-world settings. Ding et al. [12] share similar insights regarding data quality in vulnerability datasets; however, they maintained data quality through heuristic-based manual verification and by using a vulnerability database. In contrast, our study aims to create a more realistic dataset by maintaining the ratio of vulnerable to uncertain functions and reducing label inconsistency. Moreover, we found that augmenting the dataset can improve model performance.

IX. Threats to Validity

Our study uses the Big-Vul dataset [16] to construct *Real-Vul*. It is possible that some vulnerable samples in the Big-Vul dataset are mislabelled. However, the labeled samples in the dataset were manually verified by Fan et al. [16]. Also, the *Real-Vul* dataset may not fully represent all real-world scenarios since it is constructed using only ten open-source projects. However, it is worth noting that these ten projects are well-established and popular (e.g., Chrome[6] and Linux[7] and have been extensively studied and utilized in previous research [4], [54]. In *Real-Vul* dataset, we labeled all the unchanged functions of a repository as uncertain. It is possible that undiscovered vulnerabilities may exist in these functions. However, we are limited by the lack of the knowledge required to discover undiscovered vulnerabilities and the threat is present in prior datasets as well [4], [16], [54]. In RQ_2, we use balanced datasets where the number of vulnerable and uncertain samples is equal. However, the random selection of samples for the balanced datasets may impact our results since different random selections can lead to different findings [22]. To address this issue, it is recommended to train the models multiple times with various sample sets and examine

[6]https://chromium.googlesource.com/chromium/src/
[7]https://github.com/torvalds/linux

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:03:49 UTC from IEEE Xplore. Restrictions apply.

the outcomes. Unfortunately, this approach was not feasible due to our constrained computational resources.

In our study, we excluded methods that have identical MD5 hashes but conflicting labels (label inconsistency). Despite this, our dataset might still have subtle label inconsistencies that arise from variations in whitespace or new lines, which are challenging to detect. To mitigate this, we employed a regex-based strategy to merge multiple instances of new lines and whitespace into single occurrences. Nonetheless, there may be scenarios where this method falls short.

Furthermore, we focus our work on assessing the efficacy of deep learning-based techniques, which can limit the generalizability of our findings to other techniques. Future works should consider evaluating other techniques like static and dynamic analysis tools on the *Real-Vul* dataset and analyze the results.

X. Implication

Below, we distill the implication of our findings for the development of research communities.

The *Real-Vul* dataset can be used to develop tools to assist developers in mitigating vulnerabilities. For example,

- **Facilitating research studies for proposing risk assessment and prioritization techniques.** The dataset contains severity information for each vulnerable sample. This data can be used to train a model to predict the presence of vulnerabilities and assess the potential risk or severity. This aids in prioritizing security efforts and resource allocation.
- **Supporting research on automated patch generation.** The dataset contains before-fix and after-fix versions of vulnerable functions. This feature can inform the development of automated patch-generation tools.
- **Characterizing the evolution of software with respect to its vulnerabilities.** The dataset contains vulnerability data of 10 common and popular projects that span ten years of history. It contains 270,919 samples, of which 5,528 are vulnerable. Future research could explore whether severe vulnerabilities have become more or less prevalent over time or how coding practices have evolved to mitigate vulnerabilities in these projects.

XI. Conclusion

In this paper, we study the performance of deep learning-based vulnerability detection models in realistic vulnerability detection settings. First, we create a new comprehensive, realistic vulnerability detection dataset, called *Real-Vul*. *Real-Vul* contains complete source code samples of ten diverse real-world open-source projects. Then, we evaluate four *state-of-the-art* models, LineVul, DeepWukong, ReVeal, and IVDetect on the *Real-Vul* dataset. Our evaluation indicates a considerable decrease in model performance, as evidenced by a drop in precision and F1 scores of up to 95 and 91 percentage points, respectively. Our investigation also reveals that the embeddings generated by these models depict a substantial overlap between vulnerable and uncertain samples. This suggests that such models struggle to differentiate between vulnerable and uncertain samples in the *Real-Vul* dataset, resulting in a high number of false positives. Finally, we observe fluctuations in model performance based on vulnerability characteristics (e.g., vulnerability types and severity). Our study argues that when it comes to identifying vulnerabilities in realistic vulnerability detection settings, things may not be as good as they seem, and there is a need for improved model design and evaluation approaches to achieve more accurate vulnerability detection performance.

References

[1] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[2] C. Cangea, P. Veličković, N. Jovanović, T. Kipf, and P. Lio, "Towards sparse hierarchical graph classifiers," 2018, *arXiv:1811.01287*.

[3] P. Chakraborty, M. Alfadel, and M. Nagappan, "RLocator: Reinforcement learning for bug localization," 2023, *arXiv:2305.05586*.

[4] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep learning based vulnerability detection: Are we there yet," *IEEE Trans. Softw. Eng.*, vol. 48, no. 9, pp. 3280–3296, Sep. 2022.

[5] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, no. 1, pp. 321–357, Jun. 2002.

[6] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2016, pp. 785–794.

[7] X. Cheng, H. Wang, J. Hua, G. Xu, and Y. Sui, "DeepWukong: Statically detecting software vulnerabilities using deep graph neural network," *ACM Trans. Softw. Eng. Methodol.*, vol. 30, no. 3, pp. 1–33, 2021.

[8] X. Cheng et al., "Static detection of control-flow-related vulnerabilities using graph embedding," in *Proc. 24th Int. Conf. Eng. Complex Comput. Syst. (ICECCS),* Piscataway, NJ, USA: IEEE Press, 2019, pp. 41–50.

[9] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," 2014, *arXiv:1412.3555*.

[10] "CWE - CWE-888: Software fault pattern (SFP) clusters (4.12)." MITRE Corporation. Accessed: Jul. 28, 2023. [Online]. Available: https://cwe.mitre.org/data/definitions/888.html

[11] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," 2023, *arXiv:2305.14314*.

[12] Y. Ding et al., "Vulnerability detection with code language models: How far are we?" 2024, *arXiv:2403.18624v2*.

[13] A. Q. Jiang et al., "Mixtral of experts," 2024, *arXiv:2401.04088v1*.

[14] B. Rozière et al., "Code llama: Open foundation models for code," 2024, *arXiv:2308.12950v3*.

[15] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288v2*.

[16] J. Fan, Y. Li, S. Wang, and T. N. Nguyen, "AC/C++ code vulnerability dataset with code changes and CVE summaries," in *Proc. 17th Int. Conf. Mining Softw. Repositories*, 2020, pp. 508–512.

[17] Z. Feng et al., "CodeBERT: A pre-trained model for programming and natural languages," 2020, *arXiv:2002.08155*.

[18] Y. Freund et al., "Experiments with a new boosting algorithm," in *Proc. Int. Conf. Mach. Learn.,* vol. 96, 1996, pp. 148–156.

[19] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, Oct. 2001.

[20] M. Fu and C. Tantithamthavorn, "LineVul," in *Proc. 19th Int. Conf. Mining Softw. Repositories*, New York, NY, USA: ACM, May 2022, pp. 608–620.

[21] D. Grahn and J. Zhang, "An analysis of C/C++ datasets for machine learning-assisted software vulnerability detection," in *Proc. Conf. Appl. Mach. Learn. Inf. Secur.*, 2021, pp. 218–237.

[22] M. T. Hagan, H. B. Demuth, and M. Beale, *Neural Network Design*. Boston, MA, USA: PWS-Kent, 1997.

[23] M. A. Hearst, S. T. Dumais, E. Osuna, J. Platt, and B. Scholkopf, "Support vector machines," *IEEE Intell. Syst. their Appl.*, vol. 13, no. 4, pp. 18–28, Jul./Aug. 1998.

[24] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

[25] M. Iraqi, "Comparing the performance of LLMs: A deep dive into roberta, Llama 2, and mistral for disaster tweets analysis with Lora,"

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:03:49 UTC from IEEE Xplore. Restrictions apply.

Nov. 2023. GitHub. [Online]. Available: https://huggingface.co/blog/Lora-for-sequence-classification-with-Roberta-Llama-Mistral

[26] J. M. Joyce, *Kullback-Leibler Divergence*. Berlin, Germany: Springer, 2011, pp. 720–722.

[27] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2016, *arXiv:1609.02907*.

[28] Y. LeCun et al., "Convolutional networks for images, speech, and time series." in *The Handbook of Brain Theory and Neural Networks*, M. A. Arbib, Ed. MIT Press, 1995.

[29] J. Li, P. He, J. Zhu, and M. R. Lyu, "Software defect prediction via convolutional neural network," in *Proc. IEEE Int. Conf. Softw. Qual., Rel. Secur. (QRS),* Piscataway, NJ, USA: IEEE Press, 2017, pp. 318–328.

[30] Y. Li, S. Wang, and T. N. Nguyen, "Vulnerability detection with fine-grained interpretations," in *Proc. 29th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, in ESEC/FSE '21, New York, NY, USA: ACM, Aug. 2021.

[31] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, and Z. Chen, "SySeVR: A framework for using deep learning to detect software vulnerabilities," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 4, pp. 2244–2258, Jul./Aug. 2022.

[32] Z. Li et al., "VulDeePecker: A deep learning-based system for vulnerability detection," 2018, *arXiv:1801.01681*.

[33] G. Lin, S. Wen, Q.-L. Han, J. Zhang, and Y. Xiang, "Software vulnerability detection using deep neural networks: A survey," *Proc. IEEE*, vol. 108, no. 10, pp. 1825–1848, Oct. 2020.

[34] F. Lomio, E. Iannone, A. De Lucia, F. Palomba, and V. Lenarduzzi, "Just-in-time software vulnerability detection: Are we there yet?" *J. Syst. Softw.*, 2022, Art. no. 111283.

[35] N. Mehrabi, F. Morstatter, N. Saxena, K. Lerman, and A. Galstyan, "A survey on bias and fairness in machine learning," *ACM Comput. Surv.*, vol. 54, no. 6, pp. 1–35, Jul. 2019.

[36] C. Na, Y. Choi, and J.-H. Lee, "DIP: Dead code insertion based black-box attack for programming language model," in *Proc. 61st Annu. Meeting Assoc. Comput. Linguistics (Vol. 1: Long Papers)*. Toronto, Canada. Toronto, Canada: Association for Computational Linguistics, Jul. 2023, pp. 7777–7791.

[37] S. Neuhaus and T. Zimmermann, "The beauty and the beast: Vulnerabilities in Red Hat's packages." in *Proc. USENIX Annu. Tech. Conf.*, 2009, pp. 527–538.

[38] "NIST software assurance reference dataset." National Institute of Standards and Technology. Accessed: Jul. 28, 2023. [Online]. Available: https://samate.nist.gov/SARD

[39] L. E. Peterson, "K-nearest neighbor," *Scholarpedia*, vol. 4, no. 2, 2009, Art. no. 1883.

[40] J. R. Quinlan, "Learning decision tree classifiers," *ACM Comput. Surv.,* vol. 28, no. 1, pp. 71–72, 1996.

[41] A. M. M. Rahman, W. Yin, and G. Wang, "Data augmentation for text classification with EASE," in *Proc. 6th Int. Conf. Natural Lang. Speech Process. (ICNLSP)*, M. Abbas and A. A. Freihat, Eds., Association for Computational Linguistics, Dec. 2023, pp. 324–332.

[42] S.-A. Rebuffi, S. Gowal, D. A. Calian, F. Stimberg, O. Wiles, and T. Mann, "Data augmentation can improve robustness," 2021, *arXiv:2111.05328v1*.

[43] M. T. Ribeiro, S. Singh, and C. Guestrin, ""Why should I trust you?": Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, San Francisco, CA, USA, 2016, pp. 1135–1144.

[44] R. Russell et al., "Automated vulnerability detection in source code using deep representation learning," in *Proc. 17th IEEE Int. Conf. Mach. Learn. Appl. (ICMLA),* Piscataway, NJ, USA: IEEE Press, 2018, pp. 757–762.

[45] R. Scandariato, J. Walden, A. Hovsepyan, and W. Joosen, "Predicting vulnerable software components via text mining," *IEEE Trans. Softw. Eng.*, vol. 40, no. 10, pp. 993–1006, Oct. 2014.

[46] L. Van der Maaten and G. Hinton, "Visualizing data using t-SNE." *J. Mach. Learn. Res.*, vol. 9, no. 11, pp. 2579–2605, 2008.

[47] J. Walden, "The impact of a major security event on an open source project," in *Proc. 17th Int. Conf. Mining Softw. Repositories*., New York, NY, USA: ACM, Jun. 2020.

[48] Y. Wan, W. Zhao, H. Zhang, Y. Sui, G. Xu, and H. Jin, "What do they capture?" in *Proc. 44th Int. Conf. Softw. Eng.,* New York, NY, USA: ACM, May 2022.

[49] Y. Wang, G. Huang, S. Song, X. Pan, Y. Xia, and C. Wu, "Regularizing deep networks with semantic data augmentation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 7, pp. 3733–3748, Jul. 2022.

[50] F. F. Xu, U. Alon, G. Neubig, and V. J. Hellendoorn, "A systematic evaluation of large language models of code," 2022, *arXiv:2202.13169v3*.

[51] H. Yan, Y. Sui, S. Chen, and J. Xue, "Machine-learning-guided typestate analysis for static use-after-free detection," in *Proc. 33rd Annu. Comput. Secur. Appl. Conf.*, 2017, pp. 42–54.

[52] W. Zheng et al., "The impact factors on the performance of machine learning-based vulnerability detection: A comparative study," *J. Syst. Softw.*, vol. 168, 2020, Art. no. 110659.

[53] Y. Zheng et al., "D2A: A dataset built for AI-based vulnerability detection methods using differential analysis," in *Proc. IEEE/ACM 43rd Int. Conf. Softw. Eng.: Softw. Eng. Pract. (ICSE-SEIP),* Piscataway, NJ, USA: IEEE Press, 2021, pp. 111–120.

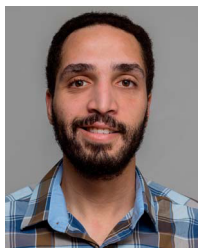
[54] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019.

Partha Chakraborty (Student Member, IEEE) is currently working toward the Ph.D. degree with Cheriton School of Computer Science, University of Waterloo, Canada. His research interests include bug localization, vulnerability detection, and the use of machine learning techniques in software engineering. For more information, see https://parthac.me/.

Krishna Kanth Arumugam received the master's degree from Cheriton School of Computer Science, University of Waterloo. His research interests include machine learning, security vulnerability detection, and mining software repositories.

Mahmoud Alfadel (Member, IEEE) is a Postdoctoral Researcher with Cheriton School of Computer Science, University of Waterloo. His research interests include mining software repositories, empirical software engineering, software ecosystems, and release engineering. For more information, see https://rebels.cs.uwaterloo.ca/member/mahmoud.html.

Meiyappan Nagappan is an Associate Professor with Cheriton School of Computer Science, University of Waterloo. He has worked on empirical software engineering to address software development concerns and currently researches the impact of large language models on software development.



Shane McIntosh (Senior Member, IEEE) is an Associate Professor with Cheriton School of Computer Science, University of Waterloo, where he leads the Software Repository Excavation and Build Engineering Labs (Software REBELs). His research interests include empirical methods to study software build systems, devops pipelines, and software quality. For more information, see https://rebels.cs.uwaterloo.ca/.

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:03:49 UTC from IEEE Xplore. Restrictions apply.